Model	Role	Latency (ms)	Decision Ratio		Decision/s
			8B	70B	
Llama3-8B-Instruct	Generator	9.8	-	-	-
Llama3-70B-Instruct	Generator	99.2	-	-	-
Gemma3-StreamGuard-0.3B	Guardrail	2.4	0.243	0.024	420
Gemma3-StreamGuard-1B	Guardrail	3.7	0.375	0.037	271
Qwen3-StreamGuard-0.6B	Guardrail	4.3	0.439	0.043	232
Qwen3-StreamGuard-1.7B	Guardrail	4.9	0.501	0.050	203
Llama3-StreamGuard-1B	Guardrail	3.2	0.330	0.033	309
Llama3-StreamGuard-3B	Guardrail	6.0	0.616	0.061	165
Llama3-StreamGuard-8B	Guardrail	9.5	0.969	0.096	105

Table 12: Latency measurements in the steady-state decoding regime. We measure from a prefix length of 1024 tokens and average over the next 1024 decoding steps, using 100 runs per setup. All experiments use NVIDIA H100 GPUs with HuggingFace and `StaticCache`; Llama3-70B-Instruct is measured on two GPUs. Decision ratios normalize guardrail latency by the average per-token latency of the Llama3-8B-Instruct and Llama3-70B-Instruct generators and are the primary quantity of interest for pipelined moderation.

are served. For completeness, we also distinguish this setting from a simpler *buffered* setup, but our interpretation and conclusions focus on the pipelined regime.

All measurements use NVIDIA H100 GPUs with the HuggingFace transformers library generation stack with StaticCache Wolf et al. (2020). The Llama3-70B-Instruct generator is measured on two GPUs; all other models are measured on a single GPU. We measure latency in a steady-state regime by starting from a prefix of 1024 tokens and timing the next 1024 generated tokens. All reported results are averaged over 100 runs. We use Llama3-8B-Instruct and Llama3-70B-Instruct as generators.

For each generator, we report the average latency per emitted token, denoted $\bar{g}$. For each guardrail, we report the average decision latency r in milliseconds and the corresponding throughput in decisions per second. To summarize pipelined performance, we additionally report the *decision ratio*

$$\rho = \frac{r}{\bar{g}},$$

computed separately against each generator. This ratio normalizes guardrail latency by the generator's average per-token latency and directly captures whether moderation keeps pace with decoding.

In the pipelined setting, ρ is the main quantity of interest. Absolute latency remains useful because it exposes raw runtime directly, but the deployment implication depends on latency relative to the generator rather than latency in isolation. A guardrail with a few milliseconds of latency may be comfortably faster than one generator and much closer to the limit for another. The decision ratio captures this dependence directly.

The decision ratio also determines post-decision exposure in the pipelined regime. Under an average-cadence approximation, the additional number of tokens served after a block decision becomes available is

$$L_{\text{extra}} = \max\left(0, \lceil \rho \rceil - 1\right) = \max\left(0, \left\lceil \frac{r}{\bar{g}} \right\rceil - 1\right),$$

with the smoother approximation $L_{\text{extra}} \approx \max(0, \rho - 1)$. In particular, when $\rho < 1$, the guardrail decision arrives before the next token would be served, so the next token does not reach the user. When $1 < \rho \leq 2$, roughly one additional token may be served before intervention, and larger values correspond to proportionally larger exposure. Thus, ρ summarizes both throughput compatibility and post-decision exposure for pipelined moderation.

For completeness, we also distinguish pipelined execution from a buffered setup. In a buffered setup, generated tokens are briefly held before display, so the generator does not pause for every guardrail decision. The resulting latency is therefore not a strict per-token sum of generation and guardrail time. If the generator is faster than the guardrail, buffering mainly affects the initial release of output, after which visible tokens are emitted at approximately the guardrail's rate. If the guardrail is faster than the generator, the two stages overlap and the added delay is correspondingly smaller. We include this setup only as a reference point; the main analysis in this section concerns pipelined moderation.

Table 12 reports the measured results. Llama3-8B-Instruct runs at 9.8 ms/token, while Llama3-70B-Instruct runs at 99.2 ms/token. Across the evaluated guardrails, decision latency ranges from 2.4 ms to 9.5 ms, corresponding to 105–420 decisions/s. Relative to the 8B generator, decision ratios range from 0.243 to 0.969. Relative to the 70B generator, they range from 0.024 to 0.096. Thus, for both generators, all evaluated guardrails satisfy $\rho < 1$. Under the average-cadence interpretation above, this means that once a block decision is available, the next token is not served to the user. The tightest pairing is Llama3-StreamGuard-8B with Llama3-8B-Instruct at $\rho = 0.969$, which is close to parity but still remains below this threshold. All other pairings provide additional latency headroom, especially for the 70B generator.

These results support the practical viability of pipelined streaming moderation. Even for the faster 8B generator, all guardrails keep pace on average, indicating that moderation can run ahead of exposure without becoming the throughput bottleneck in the measured regime. For the 70B generator, the margin is substantially larger, making pipelined deployment even more favorable.

Finally, the absolute latencies reported here should be viewed as measurements under a simple and reproducible baseline stack rather than as the lowest achievable deployment numbers. All experiments use HuggingFace with `StaticCache`. In practical serving deployments, specialized systems such as `vLLM` and optimized kernels can reduce both generator and guardrail latency further. We therefore expect lower absolute latencies in production settings, although the decision ratio remains the more relevant quantity for assessing whether a guardrail keeps pace with generation.